

Üdvözöllek a Problémamegoldás programozással gyakorlaton!

A mai óra célja: megismerni a kurzus működését és követelményeit, majd áttekinteni azokat az alapvető C# nyelvi elemeket, amelyekre a gyakorlati feladatok megoldásához szükségünk lesz.

A mai óra felépítése

1. Bemutakozás és elérhetőségek
2. Géptermi szabályok
3. Az órák menete és a szünet
4. Tantárgyi követelmények
5. Első C# alapfogalmak
6. Közös példák és kérdések
7. Gyakorlati feladatmegoldás



Bemutakozás

Oktató	Balázs Milán
E-mail	balazs.milan@nik.uni-obuda.hu
Telefon	+36303886583

Kommunikáció

- Kérdezni **órán bármikor lehet és érdemes**.
- Ha egy feladat vagy fogalom nem világos, jelezzék — a gyakorlat célja nem a „kód bemagolása”, hanem a **gondolkodásmód megértése**.
- E-mailben lehetőleg a tárgyhoz kapcsolódó, konkrét kérdésekkel keressenek.



Fontos: programozást leginkább programozással lehet megtanulni. Az órán ezért a rövid elméleti áttekintést mindig gyakorlati munka követi.



Gépterem használati szabályok

A gépteremben mindenki számára biztonságos és zavartalan munkakörnyezetet szeretnénk fenntartani.

✔ Megengedett	❌ Nem megengedett
Saját laptop használata a gyakorlatokon	A számítógépek kábelezésének megbontása
Az egyetemi gépek használata	Étkezés a gépteremben
Kérdés, közös gondolkodás, segítségkérés	A gépek fizikai konfigurációjának módosítása

⚠ Számonkérésen saját laptop nem használható.

Étkezés

Étkezni kizárólag a géptermen kívül lehet.

Italt úgy helyezzetek el, hogy az ne veszélyeztesse a számítógépeket és egyéb eszközöket.

Az órák menete

A gyakorlatokon az elmélet és a feladatmegoldás egymásra épül.

Javasolt ritmus

Rövid elmélet / demonstráció

↓

Közös példa

↓

Önálló vagy páros feladatmegoldás

↓

☕ Szünet félidőben — kb. 10–15 perc

↓

További gyakorlás és nehezebb feladatok

↓

Óra végi összefoglalás

Ha mindenkinek megfelel, **egy szünetet tartunk félidőben, körülbelül 10–15 percben.**

Tantárgyi követelmények

A tárgy teljesítése két fontos lépcsőből áll:

1. Aláírás megszerzése

A gyakorlati követelmények teljesítésével szerezhető meg az **aláírás**.

2. Vizsgára jelentkezés

Vizsgára csak megszerzett aláírással lehet jelentkezni.

Az aláírás megszerzésének feltételei

- **Rendszeres és aktív részvétel** a gyakorlatokon.
- Legfeljebb **30% hiányzás** megengedett.
- **Két évfolyamdolgozat** megírása:
 - **1. dolgozat:** 6. hét
 - **2. dolgozat:** 13. hét
- A **második dolgozaton legalább 50%** teljesítése.
- A félév során elérhető összpontszám **legalább 50%-ának** megszerzése.

 A gyakorlati eredmény beleszámít a végső jegybe.

A hivatalos és részletes követelmények mindig a tantárgyi követelményrendszerben találhatóak meg (TTK).

Feladat típusok

Sötét kék: Órán megoldandó feladatok

Világos kék: Otthoni munkára javasolt feladatok

Narancs: Nehezebb feladatok, ha már van tapasztalatuk a C# programozásban és az órán megoldják ezeket a feladatokat és bemutatják, akkor haza mehetnek.

Mit jelent programozni?

Egy program lényegében **utasítások sorozata**.

A számítógép ezeket az utasításokat meghatározott sorrendben hajtja végre.

Bemenet → Feldolgozás → Döntés / Számítás → Kimenet

A programozó feladata:

1. megérteni a problémát,
2. kisebb lépésekre bontani,
3. meghatározni a szükséges adatokat,

4. leírni a megoldás logikáját,
5. ezt a logikát programkóddá alakítani,
6. ellenőrizni, hogy a program valóban azt csinálja-e, amit szeretnénk.

💡 A jó programozó nem attól jó, hogy minden utasítást fejből tud, hanem attól, hogy **képes egy problémát pontos, végrehajtható lépésekre bontani.**

C# alapok — utasítások

A C# programok egyszerű utasításai általában **pontosvesszővel (;) végződnek.**

```
Console.WriteLine("Hello, World!");
```

A pontosvessző azt jelzi, hogy az adott utasítás véget ért.

Több utasítás

```
Console.WriteLine("Első sor");  
Console.WriteLine("Második sor");  
Console.WriteLine("Harmadik sor");
```

A program ezeket alapértelmezés szerint **felülről lefelé** hajtja végre.

Kommentek

A komment olyan szöveg a forráskódban, amelyet a program **nem hajt végre.**

Egysoros komment:

```
// Ez egy komment.  
Console.WriteLine("Ez viszont végrehajtott.");
```

Kommentet használhatunk például:

- magyarázatokhoz,
- emlékeztetőkhöz,
- ideiglenesen kikapcsolt kódrészekhez.

```
// Console.WriteLine("Ez most nem fog lefutni.");
```

✓ A komment az embernek szól, nem a számítógépnek.

Kiírás a konzolra

A konzolos programokban gyakran szeretnénk információt megjeleníteni a felhasználónak.

Console.Write()

Kiír valamit, de **nem kezd új sort**.

```
Console.Write("Hello ");  
Console.Write("World!");
```

Eredmény:

```
Hello World!
```

Console.WriteLine()

Kiír valamit, majd **új sort kezd**.

```
Console.WriteLine("Hello");  
Console.WriteLine("World!");
```

Eredmény:

```
Hello  
World!
```

Metódusok és paraméterek

A `Write` és a `WriteLine` **metódusok**.

Egy metódus neve után zárójelek következnek:

```
Console.WriteLine("Hello!");
```

A zárójelék között adjuk meg a módszernek átadott adatokat, vagyis a **paramétereket / argumentumokat**.

Ebben a példában:

```
"Hello!"
```

az az adat, amelyet a `WriteLine` ki fog írni.

Paraméter nélküli hívás

```
Console.WriteLine();
```

Ez egyszerűen egy üres sort ír ki.

Adatbekérés

A felhasználótól a `Console.ReadLine()` segítségével olvashatunk be adatot.

```
Console.Write("Mi a neved? ");  
string nev = Console.ReadLine();  
  
Console.WriteLine("Szia, " + nev + "!");
```

⚠ A `Console.ReadLine()` eredménye **szöveg** (`string`).

Ez később fontos lesz, amikor például számot szeretnénk bekérni.

Változók

A változó egy **névvel ellátott hely**, ahol adatot tárolunk.

1. Deklaráció

Megadjuk a változó **típusát** és **nevét**.

```
int x;
```

Ez azt jelenti:

„Szeretnék egy `x` nevű változót, amely egész számot tárol.”

2. Értékadás

```
x = 1;
```

Az `=` az **értékadó operátor**.

A jobb oldali érték bekerül a bal oldalon lévő változóba.

```
x ← 1
```

3. Deklaráció és értékadás egyszerre

```
int y = 2;
```

Ez rövidebb, és nagyon gyakori forma.

4. A változó értékének felhasználása

```
Console.WriteLine(x);
```

Itt nem az `"x"` szöveget írjuk ki, hanem az `x` változó **aktuális értékét**.

Fontos különbség

```
Console.WriteLine(x);
```

és

```
Console.WriteLine("x");
```

nem ugyanaz.

- `x` → a változó értéke
- `"x"` → maga az `x` karakterből álló szöveg



Alapvető változótípusok

A változó típusa meghatározza, milyen adatot tárolhatunk benne.

Típus	Mire használjuk?	Példa
int	egész számok	123 , -5 , 0
double	valós / lebegőpontos számok	1.23456 , 3.14
bool	logikai érték	true , false
char	egyetlen karakter	'A' , '\n'
string	szöveg / karakterlánc	"Hello"

Példák

```
int egesz = 123;           // 32-bites előjeles egész
double valos = 1.23456;    // 64-bites lebegőpontos szám
bool logikai = true;       // logikai érték
char karakter = '\n';       // egyetlen karakter
string karakterlanc = "Vincent! We happy?"; // szöveg
```



Literálok

A forráskódba közvetlenül beírt konkrét értékeket **literáloknak** nevezzük.

```
123
1.23456
true
'A'
"Hello"
```

A formájukból gyakran következtetni lehet a típusukra.

Figyeljünk az idézőjelekre!

```
char betu = 'A';
string szoveg = "A";
```

- 'A' → egyetlen karakter
- "A" → szöveg



Szöveg és számok átalakítása

Mivel a `Console.ReadLine()` szöveget ad vissza, numerikus feladatoknál gyakran át kell alakítani a beolvasott adatot.

```
string szoveg = Console.ReadLine();  
int szam = int.Parse(szoveg);
```

Rövidebben:

```
int szam = int.Parse(Console.ReadLine());
```

Valós szám esetén:

```
double ertek = double.Parse(Console.ReadLine());
```

Szám átalakítása szöveggé

```
double szam = 123.456;  
string szoveg = szam.ToString();
```

⚠ A `Parse` megfelelő formátumú szöveget vár. Hibás bemenet esetén a program futási hibával leállhat. A robusztusabb `adatok` bekéréssel később foglalkozunk.



Típuskonverzió röviden

Bizonyos konverziókat a `C#` automatikusan elvégéz.

```
int egesz = 13;  
double valos = egesz;
```

Itt nincs információvesztés.

Más esetekben nekünk kell jelezni a konverziót:

```
double valos = 13.5;  
int egesz = (int)valos;
```

Az eredmény:

A tizedes rész elveszik.

✖ Ezért mindig gondoljuk végig: **milyen típusú adatot szeretnénk tárolni, és elveszhet-e információ az átalakítás során?**

Elágazások — döntés a programban

Eddig a program minden utasítást egymás után hajtott végre.

Sok esetben viszont **döntést kell hoznunk**.

Például:

Ha a szám negatív → írjuk ki, hogy negatív.

Ha nulla → írjuk ki, hogy nulla.

Egyébként → pozitív.

```
int szam = -13;

if (szam < 0)
{
    Console.WriteLine("Negatív érték.");
}
else if (szam == 0)
{
    Console.WriteLine("Nulla.");
}
else
{
    Console.WriteLine("Pozitív érték.");
}
```

Hogyan működik az if ?

Általános forma:

```
if (feltétel)
{
    // Ez akkor fut le, ha a feltétel igaz.
}
```

Két ág:

```
if (feltetel)
{
    // igaz ág
}
else
{
    // hamis ág
}
```

Több lehetőség:

```
if (elsőFeltetel)
{
    // ...
}
else if (másodikFeltetel)
{
    // ...
}
else
{
    // ...
}
```

A program **az első igaz feltételhez tartozó ágot hajtja végre**.

Operátorok

Az operátorok olyan jelek, amelyek valamilyen műveletet végeznek az adatokon.

Aritmetikai operátorok

Operátor	Jelentés	Példa
+	összeadás	a + b
-	kivonás	a - b
*	szorzás	a * b
/	osztás	a / b
%	maradékképzés	a % b

Példa

```
int a = 7;
int b = 3;

Console.WriteLine(a + b); // 10
Console.WriteLine(a % b); // 1
```

Összehasonlító operátorok

Ezek eredménye mindig logikai érték: `true` vagy `false` .

Operátor	Jelentés
<code>==</code>	egyenlő
<code>!=</code>	nem egyenlő
<code><</code>	kisebb
<code><=</code>	kisebb vagy egyenlő
<code>></code>	nagyobb
<code>>=</code>	nagyobb vagy egyenlő

Például:

```
int x = 10;

Console.WriteLine(x > 5); // true
Console.WriteLine(x == 10); // true
Console.WriteLine(x != 10); // false
```

Logikai operátorok

Összetett feltételek készítésére használjuk őket.

Operátor	Jelentés
<code>!</code>	NEM
<code>&&</code>	ÉS
<code>,</code>	

ÉS — &&

```
if (eletkor >= 18 && vanJogositvany)
{
    Console.WriteLine("A feltételek teljesülnek.");
}
```

Mindkét feltételnek igaznak kell lennie.

VAGY — ||

```
if (nap == "szombat" || nap == "vasarnap")
{
    Console.WriteLine("Hétvége!");
}
```

Elég, ha legalább az egyik feltétel igaz.

NEM — !

```
if (!bejelentkezve)
{
    Console.WriteLine("Jelentkezz be!");
}
```

⚠ Az = és a == nem ugyanaz

Ez az egyik leggyakoribb kezdő hiba.

Értékadás

```
x = 5;
```

Jelentése:

x kapja meg az 5 értéket.

Összehasonlítás

```
x == 5
```

Jelentése:

„ x értéke egyenlő 5-tel?”

Az eredmény:

true vagy false

⚠ Egész osztás

Két egész szám osztásának eredménye C#-ban egész szám.

```
int a = 5;  
int b = 2;  
  
Console.WriteLine(a / b);
```

Eredmény:

2

Nem 2.5 .

Ha valós eredményt szeretnénk:

```
double a = 5;  
double b = 2;  
  
Console.WriteLine(a / b);
```

Eredmény:

2.5

💡 A változók típusa a műveletek eredményét is befolyásolja.

🧱 Kódblokkok és kapcsos zárójelek

A kapcsos zárójelek { } egy **kódblokkot** határolnak.

```
if (x > 0)
{
    Console.WriteLine("Pozitív.");
    Console.WriteLine("A feltétel igaz volt.");
}
```

A blokkban lévő utasítások együtt tartoznak az adott elágazáshoz.

✅ Már kezdettől használjunk kapcsos zárójeleket akkor is, ha az ágban csak egyetlen utasítás van.



Tipikus kezdő hibák

1. Hiányzó pontosvessző

```
int x = 5
```

❌ Hibás

```
int x = 5;
```

✅ Helyes

2. = használata == helyett

```
if (x == 5)
```

✅ Összehasonlítás

3. Rossz idézőjel

```
char betu = "A";
```

❌ char esetén egyszeres idézőjel kell:

```
char betu = 'A';
```

4. Beolvasott szám kezelése szöveggént

```
string szoveg = Console.ReadLine();
```

Ha számolni szeretnénk vele, előbb konvertálni kell.



Gyors gondolkodó kérdések

1. Mi a különbség a `Console.Write()` és a `Console.WriteLine()` között?
2. Mi a különbség az `=` és a `==` között?
3. Milyen típusban tárolnánk egy ember életkorát?
4. Milyen típusban tárolnánk egy testmagasságot méterben?
5. Mit ad vissza a `Console.ReadLine()` ?
6. Mit jelent az, hogy egy `if` feltétele `true` ?
7. Mi lesz az `5 / 2` eredménye, ha mindkét operandus `int` ?
8. Mi a különbség `'A'` és `"A"` között?



Problémamegoldási minta

Mielőtt elkezdünk kódolni, minden feladatnál tegyük fel ezt az öt kérdést:

1. Mi a bemenet?

Mit kap a program a felhasználótól?

2. Mi a kimenet?

Mit kell a programnak kiírnia vagy előállítania?

3. Milyen adatokat kell tárolnunk?

`int` ? `double` ? `string` ? `bool` ?

4. Milyen műveletekre van szükség?

Összeadás? Osztás? Maradékképzés? Összehasonlítás?

5. Kell-e döntést hozni?

Szükség van `if` / `else` szerkezetre?



Példa: gondolkodjunk kódolás előtt

Tegyük fel, hogy a programnak két szám közül a nagyobbbat kell kiírnia.

Bemenet

a
b

Döntés

a > b ?

Kimenet

a nagyobb

vagy

b nagyobb

Csak ezután kezdjük el megírni a C# kódot.



Először a probléma logikája, utána a szintaxis.

✓ Mit szeretnék, ha az óra végére magabiztosan felismernének?

- ☐ Mi az az utasítás és miért használunk pontosvesszőt?
- ☐ Hogyan írunk kommentet?
- ☐ Mi a különbség a `Write` és `WriteLine` között?
- ☐ Mit jelent egy metódushívás és mi kerül a zárójelek közé?
- ☐ Hogyan kérünk be adatot?
- ☐ Mi az a változó?
- ☐ Mi a deklaráció és az értékadás?
- ☐ Mire használjuk az `int`, `double`, `bool`, `char`, `string` típusokat?
- ☐ Hogyan alakítunk beolvasott szöveget számmá?
- ☐ Hogyan működik az `if` / `else if` / `else` ?
- ☐ Mit jelentenek a legfontosabb aritmetikai, összehasonlító és logikai operátorok?
- ☐ Mi a különbség az `=` és a `==` között?
- ☐ Hogyan bontunk egy programozási feladatot kisebb lépésekre?